Small Prompts, Big Energy and CO_2 Impact: Benchmarking Ollama LLMs on CPU and GPU

Ana Kolovska, Marjan Gusev, Dimitar Mileski

Faculty of Computer Science and Engineering, *Ss. Cyril and Methodius University*, Skopje, North Macedonia
{marjan.gushev, dimitar.mileski}@finki.ukim.mk,
ana.kolovska@students.finki.ukim.mk

***Abstract*—Energy efficiency is a critical challenge in the deployment of Large Language Models (LLMs). The consumption of electricity and associated CO_2 emissions can vary significantly depending on model architecture, parameters size, prompt length, and inference hardware. In this study, we benchmark 31 widely used Ollama models across both CPU and GPU inference, resulting in a total of 60 evaluation scenarios. Energy and carbon metrics were collected using the NVML and CodeCarbon libraries, providing insights into the environmental impact of LLM inference in data center environments.**

***Index Terms*—LLM, Ollama, Energy Efficiency, Carbon Footprint, Electricity Consumption, CPU Inference, GPU Inference, Data Centers, Environmental Impact**

I. Introduction

Large Language Models (LLMs) deployment poses growing concerns regarding energy efficiency and environmental sustainability. As model complexity increases, inference demands substantial computational resources, leading to higher electricity consumption and carbon emissions. While frameworks such as Hugging Face Transformers, TensorFlow Serving, and NVIDIA TensorRT improve scalability and throughput, fewer studies have addressed the energy and carbon implications of LLM inference. GPUs offer greater energy efficiency than CPUs due to their parallel processing capabilities; however, the environmental impact of model execution—especially within containerized and multi-hardware environments—remains insufficiently benchmarked [1]. This motivates a systematic evaluation of energy consumption and carbon footprint during LLM inference.

We evaluated 31 Ollama [2] models under two inference configurations—GPU and CPU—resulting in a total of 60 evaluation scenarios. Energy consumption and carbon footprint were analyzed across 10 different metrics collected using NVIDIA's NVML and CodeCarbon libraries, enabling a systematic assessment of energy efficiency during LLM inference.

The paper is further organized as follows. Section II reviews related work on energy-efficient LLM inference. Section III describes our experimental setup, including the selection of 31 Ollama models, the GPU and CPU configurations, and the use of NVML and CodeCarbon. Section IV presents the results from these 60 evaluation scenarios across the 10 collected metrics. Section V presents a detailed discussion of the results. Finally, Section VI concludes the paper and outlines directions for future work.

II. Related work

The increasing deployment of large language models (LLMs) has brought significant attention to their environmental and energy implications. Prior studies have highlighted the growing carbon footprint of LLM inference and training, emphasizing the importance of developing systematic methods for benchmarking energy efficiency across hardware platforms. While training efficiency has been widely studied, the inference stage, where LLMs are most frequently deployed in production remains underexplored despite being a major contributor to operational costs and emissions.

A number of research efforts have attempted to address this gap by introducing frameworks for energy-aware monitoring and benchmarking. Strubell et al. [1] first quantified the substantial carbon emissions of deep learning models, sparking further interest in measurement methodologies. More recently, CodeCarbon has been introduced as a lightweight, open-source tool to estimate emissions during machine learning workloads [3], but its reliance on generalized hardware profiles and country-level emission factors can limit its accuracy in real-world inference benchmarking. The MELODI framework [4] extends this by integrating real-time monitoring with model performance evaluation, showing that emissions correlate not only with hardware utilization but also with prompt complexity and inference latency.

Several approaches have also explored fine-grained energy tracking methods. The study [5] proposes optimizations at both the algorithmic and system level, such as dynamic batching and quantization, to reduce the energy footprint. Similarly, EcoServe [6] emphasizes energy-aware scheduling, showing that workload placement across heterogeneous hardware (CPU vs. GPU vs. accelerators) can yield significant reductions in emissions.

Another line of research has investigated real-time power measurement tools. The paper [7] argues that emission calculators such as CodeCarbon may underestimate short-lived inference tasks, as they do not capture transient energy spikes. Instead, the authors propose direct hardware-based measurements (e.g., NVIDIA's NVML for GPU monitoring), enabling more precise comparisons between CPU and GPU workloads.

Carbon-aware optimization techniques have also been discussed in related literature. The article from [8] highlights how

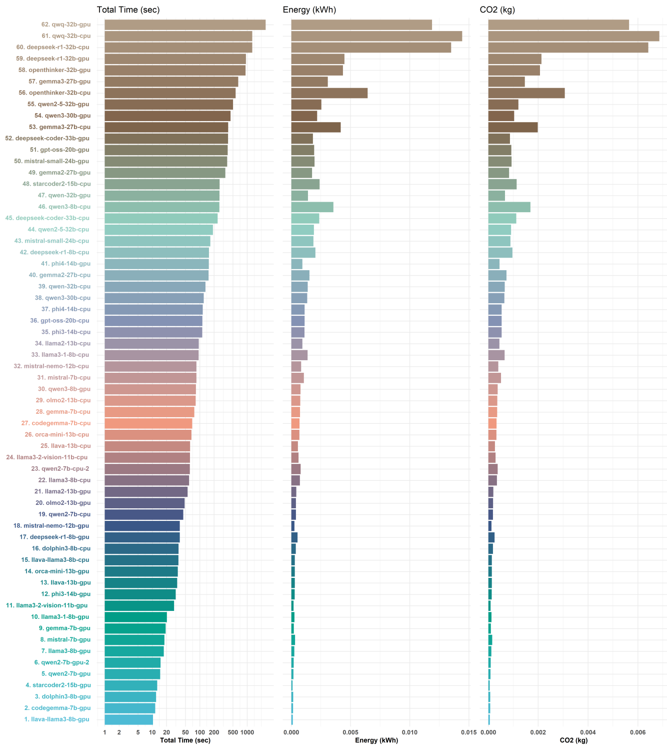


Fig. 1: NVML - Total Time (s), Energy (kWh), and CO_2 (kg) footprint comparison for all evaluated models.

workload scheduling based on renewable energy availability can reduce the net footprint of AI workloads. Additionally, [9] explores strategies such as workload deferral and carbon-intensity-based routing to improve sustainability without degrading user experience.

Finally, comparative studies of CPU versus GPU execution provide insights into the efficiency trade-offs relevant to our work. While GPUs offer higher throughput for parallelizable workloads, recent studies [10] show that CPUs can sometimes be more energy efficient for small-batch or latency-sensitive inference. Other work, including [7], highlights how hardware choice interacts with model size and request patterns to determine emissions.

Prior research has established that accurate, hardware-aware energy and emission tracking is essential for sustainable AI. Existing tools like CodeCarbon provide a foundation but are limited in capturing the dynamics of short-lived LLM inference workloads. Recent frameworks propose real-time monitoring and carbon-aware scheduling, yet few have examined these issues in the context of Ollama, a popular inference framework that supports flexible deployment on both CPU and GPU.

III. Methods

This study establishes a benchmarking framework to evaluate the **energy consumption and carbon footprint** of Ollama-based large language model (LLM) inference on both CPU and GPU. By combining *software-level estimations* with *hardware-level monitoring*, we quantify the environmental impact of inference across models of varying size and architecture.

A. Model Selection

A total of **31 LLMs available through Ollama** were selected for evaluation. These models span multiple families (e.g., LLaMA, DeepSeek, Gemma, Mistral ...) and differ in parameter scale:

- **Small to medium models**: 7B–8B parameters (optimized for lower memory footprint).
- **Intermediate models**: 14B parameters (balanced between performance and resource demand).
- **Large models**: 30B parameters (high accuracy but high computational cost).

B. Creating CPU and GPU Variants

To ensure fair comparison, we created two separate versions of each model: one configured to run on the GPU and one forced to run on the CPU. This was accomplished using **Ollama's Modelfile system**.

The **GPU version** was created from the base model using the default configuration with the Modelfile specifying `FROM [model name]` and `PARAMETER num_gpu 1`, and the command `ollama create [model name]_gpu -f /path/to/Modelfile`. The **CPU version** was created by explicitly disabling GPU allocation using `PARAMETER num_gpu 0` in the Modelfile, and the command `ollama create [model name]_cpu -f /path/to/Modelfile`.

This procedure was repeated for all 31 models, ensuring that each had a `_cpu` and `_gpu` variant. By doing so, CPU and GPU inference paths were isolated, allowing direct comparison of energy consumption and emissions under identical workloads.

C. Benchmark Prompts

To standardize evaluation, a set of prompts was designed that reflect common LLM use cases. Each experiment consisted of two phases:

- **Warm-up phase:** 3 prompts (e.g. greetings, simple factual queries) used to preload model weights into memory and stabilize runtime performance.
- **Evaluation phase:** 5 prompts covering reasoning, summarization, question answering, and creative text generation. These were designed to stress models differently depending on size and architecture.

This follows approaches in MELODI [11], where workload diversity and warm-up cycles are critical for fair energy benchmarking.

D. Execution Framework

All experiments were automated using Python scripts that launched the appropriate Ollama model (CPU or GPU variant) via the `subprocess` module. Each inference sequence (3 warm-up + 5 evaluation prompts) was run twice, once with the CPU version and once with the GPU version. Results (outputs, runtime, and hardware metrics) were logged for later analysis.

E. Energy and Carbon Measurement

Two complementary approaches were employed to capture energy and emissions:

- **CodeCarbon**: a software-based tool that estimates energy consumption and CO_2 emissions using hardware profiles and regional carbon intensity data.
- **NVIDIA NVML (via `pynvml`)**: a hardware-level monitoring library that captures real-time GPU utilization, memory usage, and power draw.

The combination of CodeCarbon and NVML enables both *emission estimation* and *fine-grained hardware monitoring*, addressing limitations of relying on a single method.

F. Comparative Analysis

For each model and prompt set, the following were recorded:

- Average CPU utilization, GPU utilization, and memory usage.
- Energy consumption in kWh (from CodeCarbon and NVML-based tracking).
- Estimated CO_2 emissions in kg CO_2eq.

Comparisons were made across:

1) CPU vs. GPU inference for the same model.
2) Model size categories (7B/8B, 14B, 30B) to assess scaling effects.
3) Prompt categories (warm-up vs. evaluation) to evaluate the relationship between task complexity and energy footprint.

This methodology enables a comprehensive analysis of the trade-offs between performance, hardware utilization, and carbon efficiency in Ollama LLM inference.

IV. Results

Fig. 1 presents the results obtained using the NVML library, while Fig. 2 shows the results from the CodeCarbon library. In all histograms, the y-axis represents the evaluated LLM models, and the x-axis corresponds to the evaluated metrics. The NVML metrics include Total Time (s), Energy (kWh), and CO_2 (kg). The CodeCarbon metrics include Duration (s), Emissions (kg), Emission Rate, Energy Consumed, GPU Energy, GPU Power, CPU Energy, and RAM Energy. A consistent color-coding scheme is applied for all models across all histograms, where each model is represented by the same color for both the y-axis label and the corresponding histogram bars.

V. Discussion

A short query like "thanks" to a large LLM typically for models above 30B consumes only 0.005–0.015 Wh of server energy, roughly equivalent to charging a small LED light for a few seconds, making a single interaction negligible. However, when scaled to billions of users (assuming 1 billion people each send one "thanks" per day at an average of 0.01 Wh per query), the daily energy usage reaches 10,000 kWh, enough to power approximately 350 average homes for a day. Over a month, this totals about 300,000 kWh, and over a year, roughly 3,650,000 kWh. With an average grid CO_2 emission of 0.4 kg per kWh, this results in around 1.46 million kg (1,460 tons) of CO_2 annually, illustrating that billions of tiny interactions can accumulate significant environmental impact. This highlights why large-scale LLM usage is monitored with tools like CodeCarbon and NVML to track energy consumption and carbon emissions. A short query like "thanks" also reinforces good conversational norms and helps people feel more connected to the technology. We can observe that in all results, both with CodeCarbon and NVML, as the model parameters increase, the total execution time, energy consumption, and CO_2 emissions also increase.

VI. Conclusion and Future Work

Benchmarking 31 Ollama models across 60 CPU and GPU scenarios with NVML and CodeCarbon shows that a short query to an LLM model above 30B consumes only 0.005–0.015 Wh of energy. Scaling to 1 billion daily queries at 0.01 Wh each results in approximately 10,000 kWh per day, 300,000 kWh per month, and 3,650,000 kWh per year, producing roughly 1.46 million kg (1,460 tons) of CO_2 annually. This illustrates how billions of tiny interactions can accumulate into a significant environmental impact and why monitoring tools like CodeCarbon and NVML are essential.

We plan to benchmark energy consumption and carbon footprint during inference to guide hardware allocation and enable more sustainable AI deployments, particularly in power-constrained edge and cloud environments.

References

[1] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in nlp," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. ACL, 2019, pp. 3645–3650.

[2] Ollama, "Ollama: Run large language models locally," 2024, [Online]. [Online]. Available: https://ollama.com

[3] CodeCarbon contributors, "Codecarbon: Track the carbon emissions of your code," https://codecarbon.io, 2022, [Online].

[4] T. Miller *et al.*, "Melodi: Multi-dimensional evaluation of llms for energy efficiency," *arXiv preprint arXiv:2407.16893*, 2024.

[5] L. Wang *et al.*, "Measuring and improving the energy efficiency of large language models inference," *ResearchGate Preprint*, 2024. [Online]. Available: https://www.researchgate.net/publication/381199506

[6] R. Kumar *et al.*, "Carbontrak: Real-time tracking of llm inference emissions," *arXiv preprint arXiv:2504.17674*, 2025.

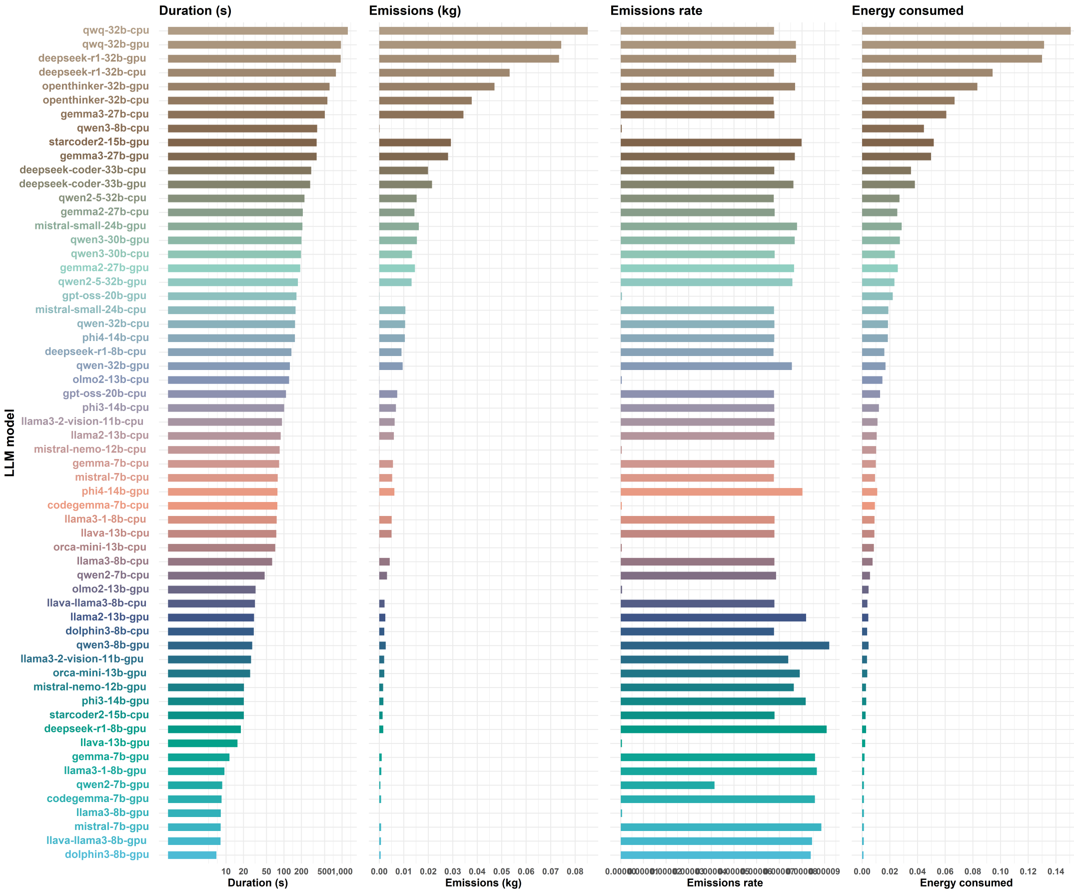
[7] M. Gonzalez *et al.*, "Measuring the environmental impact of generative ai models," *arXiv preprint arXiv:2505.09598*, 2025.

[8] W. Zhang, X. Liu, and Y. Chen, "Energy-efficient ai: Measurement, optimization, and sustainability," *Energies*, vol. 18, no. 11, p. 2810, 2025.

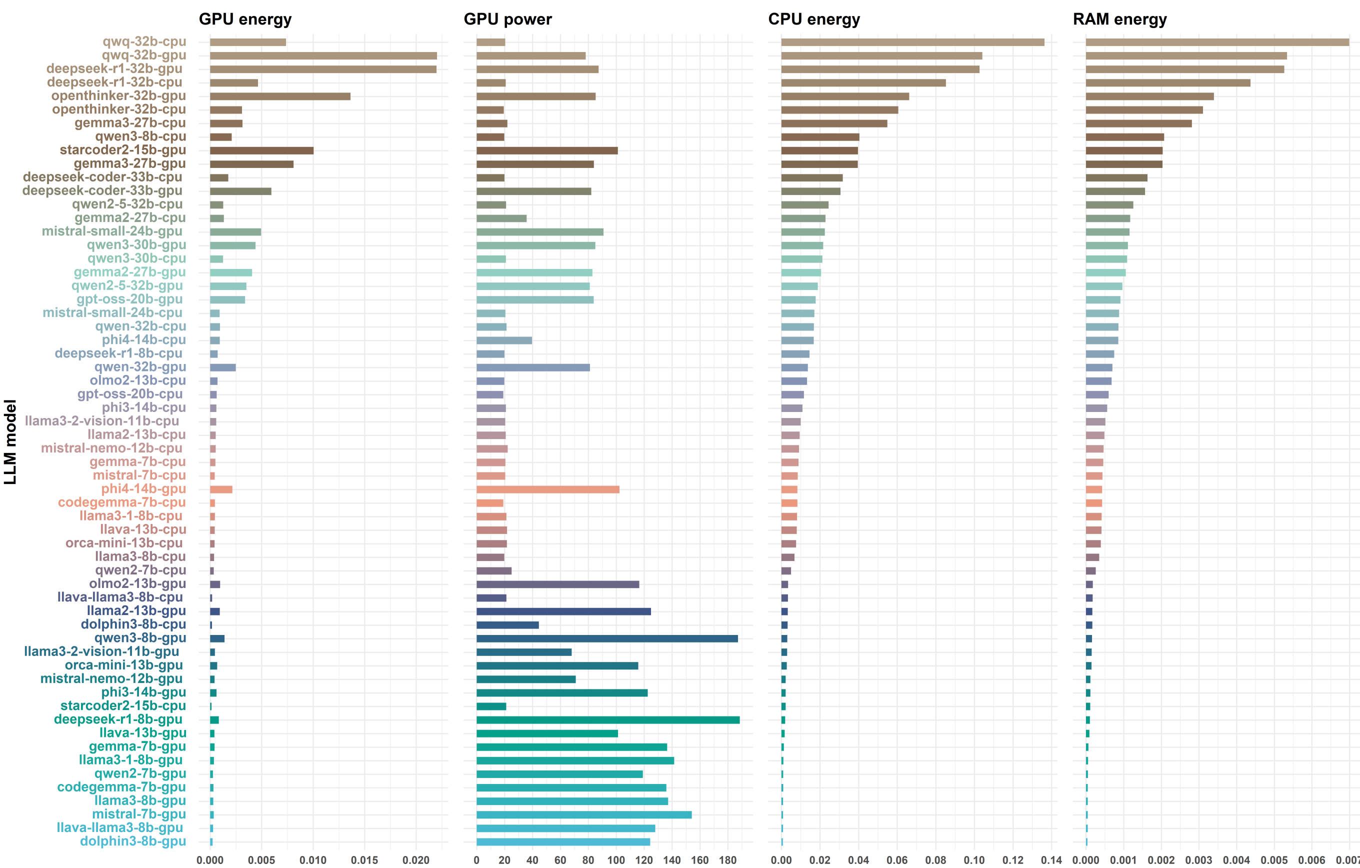
[9] A. Smith *et al.*, "Carbon-aware large language model inference: Measurement and mitigation," *arXiv preprint arXiv:2502.05610*, 2025.

[10] J. Anderson *et al.*, "Benchmarking ai energy efficiency with hybrid cpu/gpu workloads," *arXiv preprint arXiv:2504.03360*, 2025.

[11] J. Doe *et al.*, "Melodi: Benchmarking energy and carbon efficiency of llm inference," *arXiv preprint arXiv:2507.11417*, 2025.



(a) Duration, Emission, Emission Rate and Energy Consumed comparison for all evaluated models.



(b) GPU Energy, GPU Power, CPU Energy, and RAM Energy comparison for all evaluated models.

Fig. 2: Code Carbon